TP Architecture & Pilotage technique — Partie A

Jeremy Sananikone — jeremy.sananikone@supdevinci-edu.fr

Usage de l'IA

J'ai utilisé l'assistant Claude (Anthropic), via l'outil Claude Code, pour m'aider à rédiger cette partie. Il m'a servi à reformuler mes réponses afin de les rendre plus claires et à vérifier qu'elles reprenaient bien les notions et le vocabulaire du cours. En revanche, j'ai choisi moi-même les exemples (billetterie, livraison de repas, CoiffLink, location de vélos, Kafka), et j'ai relu chaque définition en la comparant au cours avant de la conserver.

Chapitre 1 — Cadrer

1. Exigence fonctionnelle vs exigence de qualité

Une exigence fonctionnelle décrit ce que le système fait, par exemple réserver un créneau ou payer. Une exigence de qualité décrit comment il le fait : sa vitesse, sa disponibilité, sa sécurité ou son coût. Ce sont les exigences de qualité qui décident de l'architecture. En effet, deux systèmes qui offrent les mêmes fonctions auront des architectures très différentes selon qu'ils doivent servir 10 ou 10 000 utilisateurs, ou rester disponibles 99,9 % du temps.

2. « L'application doit être rapide » : version testable

« Quand 5 000 spectateurs ouvrent la billetterie le jour de l'ouverture des ventes, entre 10 h et 11 h, la page de choix des places s'affiche en moins de 1 seconde pour 95 % des requêtes. »

Cette phrase précise qui est concerné (les spectateurs), à quel moment (le pic d'ouverture des ventes), sur quelle fonction (la page de choix des places) et avec quelle mesure chiffrée. Par conséquent, on peut la vérifier objectivement grâce à un test de charge, alors que « rapide » ne se mesure pas.

3. Utility tree : notes H / M / L

Chaque scénario de qualité reçoit deux notes. La première mesure l'importance pour le métier, c'est-à-dire ce que coûterait un échec à l'entreprise. La seconde mesure la difficulté technique, c'est-à-dire l'effort ou le risque nécessaire pour atteindre l'objectif.

On commence par les scénarios notés H/H, car ils sont à la fois importants et difficiles : ils structurent l'architecture et concentrent le plus de risque. Viennent ensuite les scénarios H/L, qui apportent beaucoup de valeur pour un effort faible. Les scénarios M/H peuvent être reportés, et les scénarios L/L sont des optimisations à traiter plus tard.

4. Contrainte : définition et deux exemples

Une contrainte est un élément non négociable qui limite le champ des solutions possibles avant même que la réflexion commence : un budget, un délai, les compétences de l'équipe ou une loi.

- **Compétences de l'équipe (équipe formée uniquement sur AWS)** : cette contrainte élimine d'office Azure et GCP comme fournisseurs cloud, car le temps de formation et de montée en compétence dépasserait le délai du projet.
- **RGPD (données hébergées dans l'Union européenne)** : cette contrainte élimine d'office tout hébergeur ou logiciel en ligne (SaaS) dont les serveurs se trouvent hors de l'Union européenne.

5. Le calcul au dos de l'enveloppe

Le calcul repose sur trois questions, qui s'enchaînent :

1. **Combien d'utilisateurs au total ?** On multiplie le nombre de clients par le nombre d'utilisateurs actifs par client.
2. **Quel volume de requêtes ?** On multiplie le nombre d'utilisateurs par la fréquence de leurs visites, puis par le nombre de requêtes par visite, ce qui donne un volume mensuel.
3. **Quel pic ?** On divise le volume mensuel par le nombre de secondes du mois pour obtenir une moyenne en requêtes par seconde, puis on la multiplie par un facteur de pic.

Le résultat sert à savoir si la charge est un vrai sujet. Quelques requêtes par seconde tiennent sur un seul serveur, alors que quelques centaines demandent du cache ou une capacité de montée en charge. Ce calcul identifie donc le point dur avant de choisir l'architecture, et il évite de surdimensionner ou de sous-dimensionner le système.

6. « On ne construit pas la V3, mais on la connaît »

Exemple 1 (livraison de repas) : la V3 prévoit une ouverture à d'autres pays, alors que la V1 ne couvre que la France. Dès la V1, on stocke donc les montants avec leur devise et les dates au format UTC. Cette précaution ne coûte presque rien en V1, alors que sans elle il faudrait migrer toutes les données au moment de la V3. On ne construit pas le multi-pays, mais on lui laisse la porte ouverte.

Exemple 2 (CoiffLink) : aujourd'hui, un salon est lié à un seul groupe, mais plus tard un salon pourrait dépendre de plusieurs groupes (des filiales). Dès la V1, on modélise donc la relation entre salons et groupes avec une table de liaison, plutôt qu'avec une simple clé étrangère dans la table des salons. Le surcoût est faible, et la V3 n'imposera pas de refonte du schéma.

7. Compromis : « on gagne X, on perd Y »

1. **Polling ou SSE (Server-Sent Events) pour mettre à jour l'interface** :
    - Avec le polling, **on gagne** en simplicité, car il s'agit de simples requêtes HTTP, faciles à tester et qui passent sans difficulté par les proxys. En contrepartie, **on perd** en fraîcheur et en efficacité : l'information arrive avec un délai égal à l'intervalle d'interrogation, et beaucoup de requêtes sont inutiles quand rien ne change.
    - Avec SSE, c'est l'inverse : **on gagne** des mises à jour presque instantanées avec moins de requêtes, mais **on perd** en simplicité, car le serveur doit gérer des connexions longues, les reconnexions et la montée en charge.
2. **RabbitMQ ou AWS SQS pour la messagerie** :
    - Avec RabbitMQ, **on gagne** en richesse fonctionnelle (routage fin, priorités) et on reste indépendant de tout fournisseur cloud. En contrepartie, **on perd** en charge d'exploitation, car il faut installer, superviser et mettre à jour le serveur soi-même.
    - Avec SQS, **on gagne** un service géré par AWS, donc sans maintenance, mais **on perd** en flexibilité de routage et on devient dépendant d'AWS.

Chapitre 2 — Un code qu'on peut changer

8. Couplage et cohésion

- **Le couplage** mesure à quel point deux composants dépendent l'un de l'autre. Si modifier A oblige à modifier B, les deux composants sont fortement couplés. On cherche un couplage **faible**, car chaque modification reste alors locale.

- **La cohésion** mesure à quel point le contenu d'un module forme un ensemble logique, centré sur un seul sujet. On cherche une cohésion **forte**, car un module qui ne traite qu'un sujet est plus facile à comprendre et à modifier.

9. Trois façons de dépendre l'un de l'autre

Voici les trois façons, de la pire à la meilleure :

1. **La base de données partagée** : les deux modules lisent et écrivent dans les mêmes tables. Le simple renommage d'une colonne peut alors tout casser, et personne ne voit clairement qui dépend de quoi.
2. **L'appel synchrone** : le module A appelle le module B et attend sa réponse. Par conséquent, si B tombe en panne ou ralentit, A est bloqué. Par exemple, si le service d'envoi de SMS est en panne, la réservation devient impossible.
3. **L'événement** : le module A publie un message qui signifie « ceci est arrivé », sans savoir qui l'écoute, et les autres modules réagissent chacun de leur côté.

L'événement est la meilleure solution, car l'émetteur ne connaît pas ses récepteurs. On peut donc ajouter ou retirer un récepteur sans toucher à l'émetteur, et la panne d'un récepteur ne bloque pas l'émetteur.

Exemple avec Kafka (site de e-commerce) : le service Commande publie un événement `commande.confirmée` dans un topic Kafka, puis passe à la suite sans attendre de réponse. Trois services indépendants lisent ce topic, chacun dans son propre groupe de consommateurs : Facturation génère la facture, Notifications envoie l'email et Stock décrémente les quantités. Pour ajouter un service « fidélité », il suffit de créer un nouveau consommateur, sans modifier le service Commande. De plus, si le service Notifications tombe en panne, Kafka conserve les événements : à son retour, le service reprend à son dernier offset et rattrape son retard, sans perte et sans impact sur les commandes.

10. Métier et technique

Les exemples suivants concernent une plateforme de location de vélos.

Règles métier (le dirigeant les comprend sans connaissances informatiques) :

1. Une location de plus de 24 h bénéficie de 20 % de réduction.
2. La location d'un vélo électrique exige une caution de 300 €.
3. Une réservation non honorée est annulée après 30 minutes de retard.

Briques techniques :

1. Appeler l'API de Stripe pour débiter la carte bancaire.
2. Exécuter la requête SQL qui enregistre la réservation.
3. Formater en JSON la réponse envoyée à l'application mobile.

On sépare ces deux catégories parce qu'elles changent pour des raisons différentes et à des rythmes différents. Une règle métier change sur décision du dirigeant, alors qu'une brique technique change quand on remplace un fournisseur ou une technologie. Si les deux sont mélangées dans le même code, remplacer Stripe oblige à relire les règles de réduction, avec le risque de les casser.

11. Le D de SOLID

Avec une analogie : une lampe (le métier) ne se branche pas directement sur la centrale d'un fournisseur d'électricité ; elle se branche sur une prise (l'interface). Comme cette prise accepte aussi bien EDF, TotalEnergies qu'une batterie, on peut changer de fournisseur sans toucher à la lampe.

Sans analogie : le principe d'inversion des dépendances (Dependency Inversion) signifie que le code métier dépend d'une interface, c'est-à-dire d'un contrat qui liste les méthodes promises, et jamais d'un fournisseur concret. Chaque fournisseur (Stripe, un prestataire de SMS) implémente cette interface. Le métier appelle donc toujours la même méthode, quelle que soit l'implémentation : c'est le polymorphisme, qui choisit au moment de l'exécution l'implémentation réellement utilisée. Par conséquent, changer de fournisseur revient à écrire une nouvelle implémentation, sans modifier le métier.

12. KISS, DRY, YAGNI

- **KISS** (Keep It Simple, Stupid) : on ne choisit une solution complexe que si un besoin réel et chiffré la justifie, et jamais par simple curiosité technologique.
- **DRY** (Don't Repeat Yourself) : chaque règle métier est écrite à un seul endroit du code, afin qu'une modification ne soit faite qu'une fois.
- **YAGNI** (You Aren't Gonna Need It) : on ne code pas une fonctionnalité « au cas où », mais on garde ouvertes les portes qui ne coûtent rien. Par exemple, un salon n'appartient aujourd'hui qu'à un seul groupe, mais on modélise dès la V1 la relation en N:N plutôt qu'en 1:N. Dans le code, cela revient à manipuler une liste de groupes plutôt qu'un seul groupe. Ce choix ne coûte presque rien aujourd'hui ; en revanche, il évite une refonte du schéma et du code quand les filiales arriveront, ainsi que l'écriture de scripts de migration de la base de données.

Le piège du DRY : deux règles qui ont la même valeur aujourd'hui, mais qui évoluent pour des raisons différentes, ne doivent pas être fusionnées. Prenons une boutique en ligne : « livraison offerte dès 50 € » est une décision logistique, tandis que « paiement en trois fois dès 50 € » est une décision financière. Si on les fusionne dans une seule constante, modifier l'une modifie aussi l'autre sans le vouloir, ce qui provoque des bugs.

13. Design pattern, familles, « patternite »

Un design pattern est une solution nommée et éprouvée à un problème de conception qui revient souvent. Les patterns se répartissent en trois familles :

- **Les patterns de création** expliquent comment créer les objets (Factory).
- **Les patterns de structure** expliquent comment assembler les objets entre eux (Adapter, Facade, Decorator).
- **Les patterns de comportement** expliquent comment les objets collaborent (Strategy, Observer, State, Repository).

La **patternite** désigne l'abus de patterns : on les applique par réflexe ou pour faire « propre », alors qu'aucun problème réel ne le justifie. Le résultat est un code plus complexe, avec davantage d'indirections et plus difficile à lire, sans aucun gain en retour. Un pattern doit donc toujours répondre à un problème précis dans le code ; sinon, il viole le principe KISS.

14. Les 8 patterns : le problème que chacun règle

- **Factory** : la décision « quel objet créer » (par exemple le canal d'envoi : SMS, email ou notification push) est recopiée à plusieurs endroits du code. Le pattern la centralise dans un seul endroit, qui choisit et construit le bon objet.
- **Adapter** : un fournisseur externe attend un format différent du nôtre (12,50 € chez nous, mais 1 250 centimes chez Stripe). Le pattern place un traducteur par fournisseur, afin que le métier ignore tout de ces détails.
- **Facade** : une opération exige plusieurs appels dans un ordre précis, et cette séquence est recopiée à plusieurs endroits. Le pattern expose une seule fonction simple, que tous les autres modules appellent.
- **Decorator** : on veut ajouter un comportement (un cache, de la journalisation) à un service existant sans modifier son code. Le pattern enveloppe ce service dans une classe qui expose la même interface et ajoute le comportement voulu avant de transmettre l'appel.

- **Strategy** : plusieurs variantes d'un même calcul (tarif normal, tarif étudiant, happy hour) multiplient les `if` dans tout le code. Le pattern isole chaque variante dans une classe qui expose la même méthode.
- **Observer** : un événement (une réservation confirmée) doit déclencher une liste de réactions qui ne cesse de s'allonger (SMS, statistiques, fidélité). Le pattern laisse l'émetteur annoncer l'événement, tandis que chaque module intéressé s'abonne et réagit de son côté.
- **State** : le cycle de vie d'un objet autorise certaines transitions et en interdit d'autres (par exemple, une commande expédiée ne doit pas pouvoir être remboursée). Le pattern écrit ces règles une seule fois, et l'objet refuse lui-même les transitions interdites.
- **Repository** : les requêtes d'accès aux données sont éparpillées dans de nombreux fichiers, ce qui rend impossible de tester le code sans base de données. Le pattern regroupe tout l'accès aux données dans une seule classe, que l'on peut remplacer par une version en mémoire pendant les tests.

15. Testabilité et stubs

Un test isole un morceau de code et vérifie son comportement. Si ce morceau est difficile à tester, c'est qu'il mélange plusieurs responsabilités ou qu'il dépend directement de l'extérieur (base de données, API de paiement, envoi de SMS) : pour le tester, il faut alors installer tout son environnement. À l'inverse, un code bien découpé, c'est-à-dire avec une forte cohésion, un couplage faible et des dépendances cachées derrière des interfaces, se teste seul, rapidement et sans infrastructure.

Les architectures **hexagonale (ports & adapters)** et **Clean Architecture** reposent sur cette idée par leur structure même. Le métier est placé au centre et ne dépend de rien ; il expose des **ports**, c'est-à-dire des interfaces. Les dépendances techniques (base de données, Stripe, SMS, API web) sont des **adapters** branchés à l'extérieur, et la dépendance pointe toujours vers le centre, conformément au principe D de SOLID. Par conséquent, en test, on branche des stubs sur les mêmes ports, sans toucher au métier.

Un **stub** (un service de paiement simulé, une base de données en mémoire) est un remplaçant qui respecte la même interface que la vraie dépendance, mais qui ne produit aucun effet réel. Grâce à lui, on teste le métier en moins d'une seconde, sans carte bancaire, sans envoyer de SMS et sans installer de base de données. De plus, il permet de simuler des cas difficiles à provoquer en réalité, comme un paiement refusé ou un fournisseur en panne.

Chapitre 3 — Modéliser le métier

16. Cœur, support, générique

- **Le cœur** est ce qui différencie l'entreprise de ses concurrents et fait sa valeur. Pour une application de covoiturage, c'est l'algorithme de mise en relation entre conducteurs et passagers. Il est donc développé sur mesure.
- **Le support** est nécessaire au fonctionnement, mais ne différencie pas l'entreprise (back-office, exports comptables). On le code simplement, sans chercher la perfection.
- **Le générique** est identique dans presque tous les systèmes (authentification, paiement, envoi d'emails et de SMS). On ne le code pas soi-même.

Les meilleurs développeurs doivent travailler sur le cœur, car c'est là que la valeur est créée. À l'inverse, **on achète le générique** : Auth0 pour l'authentification, Stripe pour le paiement et un prestataire spécialisé pour les emails et les SMS. Passer des semaines à coder un système de connexion maison, c'est autant de temps retiré au cœur du produit.